

JAVA COLLECTION EXCEPTIONS

A funny, hand-drawn guide from super simple to deep understanding

Meet Byte the little coder!

Imagine Java Collections as a toy box. You can put toys in, take toys out, find a toy, and sort them. But the toy box has rules. If you break a rule, Java raises an exception — like a tiny alarm bell!

Our learning adventure

We will meet six common exceptions, see the wrong code, fix it, peek inside Java's iterator, and learn which collection to choose in real backend projects.

Our promise
No memorising scary definitions first. We start with a tiny story, then look at Java code, then understand what the JVM and collection are doing.

Inside this guide: NullPointerException • IndexOutOfBoundsException • ConcurrentModificationException • UnsupportedOperationException • ClassCastException • IllegalStateException • Iterator behaviour • Backend practices



1. What is a Java Collection?

A collection is simply a container that stores a group of objects. Think of a toy box: it can hold cars, balls, dolls, and blocks. In Java, we have containers such as List, Set, and Map.

```
List<String> toys = new ArrayList<>();

toys.add("Car");
toys.add("Ball");
toys.add("Teddy");

System.out.println(toys);
```

Here, List is the type of container and ArrayList is one implementation. The list currently holds three String objects.

Three toy boxes you should know

Java type	Toy-box story	Main idea
List	A line of numbered toy slots	Keeps order; can contain duplicates.
Set	A box that refuses duplicate toys	Stores unique elements.
Map	A set of labelled drawers	Stores key-value pairs.

What is an exception?
An exception is Java's way of saying: "Something went wrong while I was running this instruction." It is like an alarm. The alarm tells us where to investigate; it does not automatically fix the problem.

Why should a backend developer care?

Collections are everywhere: API responses, shopping carts, user roles, cached data, database results, order lists, and message processing. A small collection mistake can break an API request or a background job.

Remember: The best fix is usually to understand the collection's rules and write code that follows them — not to hide every error inside a catch block.



2. NullPointerException — the invisible toy

Imagine Byte asks you to paint a toy car, but the toy box is empty. There is no car to paint! In Java, null means “there is no object here.” If you call a method on null, Java throws NullPointerException (NPE).

```
List<String> names = new ArrayList<>();
names.add("Maya");
names.add(null);

for (String name : names) {
    System.out.println(name.toUpperCase());
}
```

The first name is a real String, so toUpperCase() works. The second value is null. Java cannot make null uppercase because there is no String object.

Fix 1 — check before using

```
for (String name : names) {
    if (name != null) {
        System.out.println(name.toUpperCase());
    }
}
```

Fix 2 — reject null at the entrance

```
String safeName = Objects.requireNonNull(
    inputName, "Name cannot be null"
);

names.add(safeName);
```

Deep understanding
ArrayList allows null elements. But ConcurrentHashMap does not allow null keys or values. Different collections have different rules, so always check the contract of the collection you use.

Real backend example

Suppose an API receives a customer's name. If the name is mandatory, validate it when the request enters your application. Do not let a null value travel through five service classes and crash later.

Child-friendly memory trick: Before using a toy, first check that the toy is actually in your hand.



3. IndexOutOfBoundsException — asking for a toy slot that does not exist

Imagine a shelf with three slots. Java numbers them 0, 1, and 2 — not 1, 2, and 3. If you ask for slot 3, Java says: "There is no slot number 3!"

```
List<String> tools = List.of(
    "Eclipse", "IntelliJ", "Maven"
);

System.out.println(tools.get(2)); // Maven
System.out.println(tools.get(3)); // ERROR
```

There are three elements. Valid indexes are 0, 1, and 2. The rule is: last valid index = size() - 1.

A classic beginner mistake

```
for (int i = 0; i <= tools.size(); i++) {
    System.out.println(tools.get(i)); // Wrong
}

// Correct:
for (int i = 0; i < tools.size(); i++) {
    System.out.println(tools.get(i));
}
```

Why is <= wrong here?
If size() is 3, the loop runs with i = 0, 1, 2, and 3. But index 3 does not exist. Use i < size(), not i <= size().

Real backend example: pagination

A page of results may contain fewer records than requested. Never assume the list has the number of items you expected. Check the actual size before accessing an index.

Memory trick: Java list indexes start from zero. Count the slots, then subtract one to find the last slot.



4. ConcurrentModificationException — changing the toy box while using it

Imagine you are checking every toy in a box. While you are walking through the toys, you suddenly remove one directly from the box. Your way of tracking "which toy comes next" may no longer match the box. A fail-fast iterator can detect this and throw ConcurrentModificationException (CME).

Surprise: Despite its name, CME can happen in a single-threaded program. You do not need two threads.

Wrong code — the list is changed behind the iterator's back

```
List<String> toys = new ArrayList<>(
    List.of("Car", "Ball", "Teddy")
);

for (String toy : toys) {
    if (toy.equals("Ball")) {
        toys.remove(toy); // May throw CME
    }
}
```

The enhanced for loop uses an Iterator internally. The iterator is managing the walk through the list, but toys.remove() changes the list directly. The iterator may notice the unexpected structural change.

Fix 1 — let the iterator remove the item

```
Iterator<String> it = toys.iterator();

while (it.hasNext()) {
    String toy = it.next();

    if (toy.equals("Ball")) {
        it.remove();
    }
}
```

Why does this work?
The iterator performs the removal itself. It can keep its own tracking information in sync with the collection.



5. CME solutions — choose the right tool

Fix 2 — removeIf(), the clean Java 8+ way

Read this line like a sentence: "Remove each toy if its name is Ball." The arrow is a lambda expression; the condition returns true for an item that should be removed.

```
toys.removeIf(toy -> toy.equals("Ball"));
```

A second example — remove even numbers

```
List<Integer> numbers = new ArrayList<>(
    List.of(10, 15, 20, 25, 30)
);

numbers.removeIf(number -> number % 2 == 0);

System.out.println(numbers);
// [15, 25]
```

The percent symbol (%) gives the remainder. An even number divided by 2 has remainder 0, so 10, 20, and 30 are removed.

Fix 3 — concurrent collection for shared multi-threaded data

```
CopyOnWriteArrayList<String> toys =
    new CopyOnWriteArrayList<>(
        List.of("Car", "Ball", "Teddy")
    );

for (String toy : toys) {
    if (toy.equals("Ball")) {
        toys.remove(toy);
    }
}
```

CopyOnWriteArrayList is useful when many threads read a list and writes are relatively rare. Its iterator sees a snapshot — a copy-like view of the list as it was when the iterator was created. A later change does not alter that iterator's view.

> *Important trade-off*
> *CopyOnWriteArrayList copies its backing array when it is modified. That makes writes relatively expensive. Do not use it automatically for every list.*

ConcurrentHashMap is different: its iterators are weakly consistent. They can continue while updates happen and may reflect some updates, but they are not fixed snapshots.



6. Deep dive — what is modCount?

Let's peek inside the ArrayList toy box. ArrayList keeps an internal counter called modCount. You do not normally use this field yourself; it is an implementation detail that helps fail-fast iterators detect unexpected structural changes.

Imagine this tiny story

- *The list has modCount = 3.*
- *An iterator is created. It remembers expectedModCount = 3.*
- *Someone adds an item directly to the list. modCount becomes 4.*
- *The iterator checks its expected value: 3. The list's value is now 4.*
- *The mismatch may cause ConcurrentModificationException.*

Name	Toy-box explanation
modCount	*The collection's current change counter.*
expectedModCount	*The counter value remembered by the iterator.*
Mismatch	*The iterator may detect an unexpected structural change.*

```
if (modCount != expectedModCount) {
    throw new ConcurrentModificationException();
}
```

This is a simplified illustration of the check, not the complete JDK source code.

> *Very important interview point*
> *Fail-fast is best-effort bug detection. It is not guaranteed to catch every modification, and it is not a thread-safety mechanism. Never use CME detection as a way to coordinate threads.*



7. UnsupportedOperationException — this toy box is locked!

Imagine a display box in a shop. You can look at the toys, but the shopkeeper does not let you add or remove them. Some Java collections are read-only or fixed-size too.

```
List<String> names = List.of("Amit", "Vivek");

names.add("Rahul"); // UnsupportedOperationException
```

List.of() creates an unmodifiable list. You can read its values, but you cannot add, remove, or replace elements.

Arrays.asList() has a different rule

```
List<String> names =
    Arrays.asList("Amit", "Vivek");

names.set(0, "Anita"); // Allowed
names.add("Rahul");    // Not allowed
```

Arrays.asList() returns a fixed-size list backed by an array. Replacing an existing item is allowed, but changing the size is not.

Fix — make a new mutable box

```
List<String> names = new ArrayList<>(
    Arrays.asList("Amit", "Vivek")
);

names.add("Rahul");
names.remove("Amit");
```

> *Remember*
> *The type List does not promise that every operation is supported. Know whether your list is mutable, fixed-size, or unmodifiable.*



8. ClassCastException — trying to fit a square toy into a round hole

Every Java object has a real type. If you tell Java to treat an object as a completely different, incompatible type, Java cannot safely do that and may throw ClassCastException.

```
List rawList = new ArrayList();
rawList.add("Java");
rawList.add(100);

for (Object value : rawList) {
    String text = (String) value;
    System.out.println(text);
}
```

The first object is a String, so the cast works. The second object is an Integer. An Integer is not a String, so the cast fails.

Fix — use generics

```
List<String> languages = new ArrayList<>();

languages.add("Java");
// languages.add(100); // Compiler error
```

Generics tell the compiler what kind of objects belong in the collection. The compiler can catch many mistakes before the program runs.

> *Backend tip*
> *Avoid raw List, Set, and Map declarations. Prefer List, Set, or Map. Validate external data before casting it to a domain type.*

9. IllegalStateException — doing the right action at the wrong time



Imagine a toy robot that can only move after you press its start button. If you ask it to move before starting it, the robot says: "I am not ready!" Some Java objects also require methods to be called in a particular order.

An Iterator is a good example. You must call next() to get an item before calling remove() to remove that returned item.

```
List<String> toys = new ArrayList<>(
    List.of("Car", "Ball")
);

Iterator<String> it = toys.iterator();
it.remove(); // IllegalStateException
```

Why? The iterator has not returned an item yet. It has nothing to remove.

Correct order

```
Iterator<String> it = toys.iterator();

while (it.hasNext()) {
    String toy = it.next();

    if (toy.equals("Car")) {
        it.remove();
    }
}
```

Iterator rule
Call next() first. Then you may call remove() once for that returned element. Calling remove() before next(), or twice without another next(), is an illegal state.



10. Fail-Fast vs Snapshot vs Weakly Consistent

You may hear the phrase “fail-safe iterator” in tutorials. It is a popular informal phrase, but it is not an official Java API category. It is more accurate to describe the actual behaviour.

Behaviour	How it behaves	Example
Fail-fast	May detect unexpected structural modification and throw CME. Best-effort only.	ArrayList, HashMap iterators
Snapshot	Reads the snapshot captured when the iterator was created.	CopyOnWriteArrayList
Weakly consistent	Continues during updates and may reflect some changes; not a fixed snapshot.	ConcurrentHashMap

Which one should I use?

- Normal local list processing? ArrayList is usually fine.
- Many readers, few writers on a shared list? Consider CopyOnWriteArrayList.
- A map shared between threads? Consider ConcurrentHashMap.
- Need to remove matching elements from an ordinary mutable collection? Consider removeIf().

> Big lesson
> A concurrent collection is not automatically the best collection. Choose it only when the application really needs shared concurrent access and its consistency/performance trade-offs fit your workload.



11. Real backend example — filtering orders

Suppose a Spring Boot service loads orders and must exclude cancelled orders before returning a result. If the list is a mutable working list and it is okay to change it, removeIf() is clear and readable.

```
List<Order> orders = orderRepository.findAll();

orders.removeIf(order ->
    order.getStatus() == OrderStatus.CANCELLED
);

return orders;
```

But what if you must keep the original list unchanged? Create a new filtered result instead.

```
List<Order> activeOrders = orders.stream()
    .filter(order ->
        order.getStatus() != OrderStatus.CANCELLED)
    .toList();
```

In modern Java, Stream.toList() returns an unmodifiable list. If you need a mutable result, collect into a new ArrayList.

```
List<Order> activeOrders = orders.stream()
    .filter(order ->
        order.getStatus() != OrderStatus.CANCELLED)
    .collect(Collectors.toCollection(ArrayList::new));
```

Before shipping code, ask yourself

- *Can this list be null? Can any element be null?*
- *Am I using an index that actually exists?*
- *Is this collection mutable?*
- *Am I changing the collection while iterating?*
- *Could more than one thread access this collection?*
- *Am I using generics to protect the element type?*

12. Final revision — six exceptions in one minute

Exception	Remember it like this	Typical fix
NullPointerException	The toy is missing (null).	Validate/check null.
IndexOutOfBoundsException	That shelf slot does not exist.	Check index against size.
ConcurrentModificationException	The box changed while the iterator walked through it.	Iterator.remove(), removeIf(), or suitable concurrent collection.
UnsupportedOperationException	This box does not allow that action.	Use a mutable collection if changes are required.
ClassCastException	You treated one kind of toy as another.	Use generics and correct types.
IllegalStateException	You did the action at the wrong time.	Follow the API's method order.

Interview questions to practise

- Can ConcurrentModificationException happen in a single-threaded program?
- What is modCount and expectedModCount?
- Why is Iterator.remove() safe during iterator traversal?
- What is the difference between removeIf() and remove()?
- Why does List.of() throw UnsupportedOperationException on add()?
- How are CopyOnWriteArrayList and ConcurrentHashMap iterators different?
- How do generics reduce ClassCastException?

LinkedIn-ready closing summary

Java Collection Exceptions — understand the reason, not just the error!

Collections are used in almost every Java backend application. But each collection has rules. A null value, invalid index, unsupported operation, wrong type, or unsafe modification can cause a runtime exception.

The real skill is not memorising exception names. It is understanding why they happen, what Java is doing internally, and choosing the right solution.

Good developers do not just make the error disappear. They make the code predictable, readable, and safe.

#Java #JavaDeveloper #CollectionsFramework #SpringBoot #BackendDevelopment #Multithreading #SoftwareEngineering

One final thought
Treat every exception like a tiny teacher. It is showing you a rule your code forgot to follow. Learn the rule, fix the cause, and your Java skills get stronger!